

RAG GenAI Demo

Architecture Overview

This is a full-stack AI-powered document chat application with:

- Frontend: **React** SPA with tabbed interface for both **general chat** and **document chat** features
- Backend: **FastAPI** REST API handling document upload, embedding generation, and **RAG (Retrieval-Augmented Generation)**
- AI Pipeline: Document ingestion → text splitting → embedding generation → vector storage → semantic search → **LLM response** generation

The system enables users to upload documents (PDF/TXT), ask questions about them, and receive AI-generated answers based on the document content using local LLMs and vector similarity search.

Here's the complete technology stack for the project in this Demo:

Frontend (React Application)

- React 19.2.5 - Modern React with hooks and functional components
- Vite - Fast build tool and development server
- ESLint - Code linting and quality assurance
- Babel - JavaScript transpilation with React compiler preset
- CSS - Standard styling (no CSS frameworks)

Backend (FastAPI)

- FastAPI 0.95.2 - High-performance Python web framework
- Uvicorn 0.24.0 - ASGI server for running FastAPI
- Python-Multipart 0.0.6 - File upload handling
- Pydantic 1.10.13 - Data validation and serialization

AI/ML Components

- Ollama 0.1.0 - Local LLM inference (Llama 3 model)
- Sentence Transformers - Text embedding generation (all-MiniLM-L6-v2 model)
- LangChain 0.1.0 - LLM application framework
- LangChain Community 0.0.9 - Additional LangChain integrations

Database & Storage

- ChromaDB 0.4.10 - Vector database for document embeddings
- Local File System - Document storage in uploaded_documents

Document Processing

- PyPDF 3.17.0 - PDF document loading and parsing
- RecursiveCharacterTextSplitter (from LangChain) - Text chunking for embeddings

Development Tools

- Python Virtual Environment - Dependency isolation
- VS Code - IDE with Copilot integration
- Git - Version control













